Preparing Datasets for ML Model Training

Dr. Oybek Eraliev,

Department of Computer Engineering

Inha University In Tashkent.

Email: oybekeraliev7@gmail.com

Content

Foundations of Data
What data is, why preparation matters, structured vs. unstructured

The Preparation Pipeline
Collect, inspect, clean, transform, store, visualize

Cleaning
Missing values, duplicates, and outliers

Quality, Scaling & Encoding
Quality metrics, normalization, categorical encoding

Scale, Imbalance & Augmentation
Big data, class imbalance, SMOTE, augmentation

Feature Engineering & Selection
Building and choosing the right inputs for a model

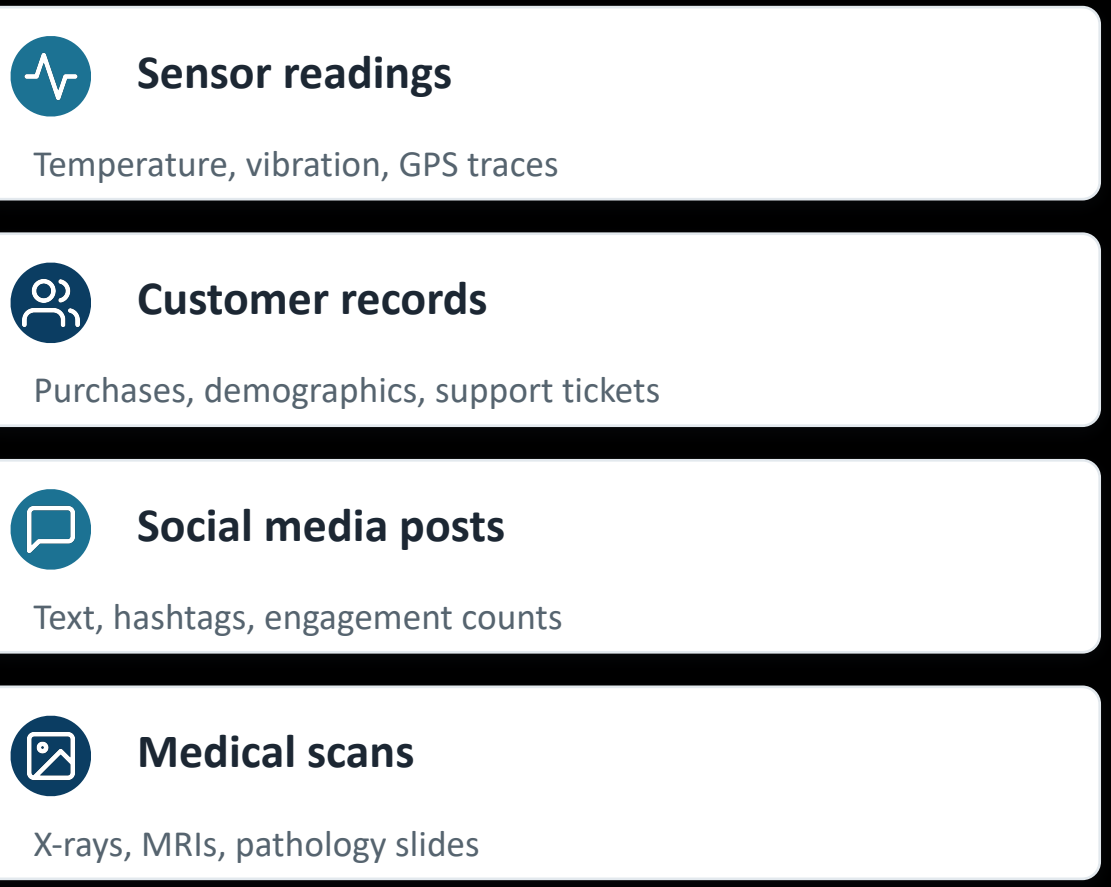
Modern Techniques
Data-centric AI, synthetic data, embeddings, validation

Best Practices & Wrap-Up
Leakage, tooling, checklist, key takeaways

What Is Data?

Data is any recorded fact, measurement, or observation that a system can process and learn from — numbers, words, images, sound, or signals captured from the real world.

For a machine learning model, data is the only source of experience — it never "sees" the world any other way.



Data Is the Fuel of the Modern Economy

2.5 EB
generated worldwide every single day
(1 exabyte = 1 billion gigabytes)

80%
of enterprise data is unstructured
or poorly organized

60–70%
of a data scientist's time goes into preparation, not modeling

$3.1T
lost yearly in the US to poor-quality
data — IBM estimate

"Garbage in, garbage out." A model can never be more reliable than the data it was trained on — which is exactly why preparation, not architecture, is where most real-world ML projects succeed or fail.

Types of Data: Structured

Structured data fits neatly into rows and columns — the easiest form to store, query, and feed directly into most models.

Tabular Data

Spreadsheets and CSVs — organized as rows and columns, one record per row. Each row is a single observation; each column, one feature.

Relational Databases

SQL, PostgreSQL, MySQL — structured tables linked by keys and constraints, queried directly to build a training table.

Time Series

Stock prices, IoT sensor streams — values indexed by timestamp, usually split chronologically rather than randomly.

Structured data is what most classic ML algorithms — linear models, trees, gradient boosting — expect directly, with little extra preprocessing.

Unstructured & Semi-Structured Data

Over 80% of newly created data is unstructured — and it demands very different preparation techniques than a spreadsheet.

Text (NLP)

Emails, articles, social posts — needs tokenization, cleaning, and embeddings.

Images & Video

JPEG, PNG, MP4 — needs resizing, normalization, and augmentation.

Audio

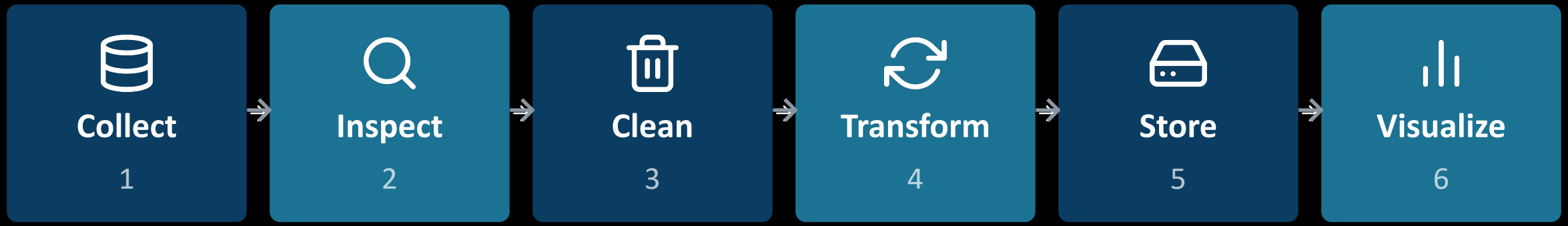
Recordings, speech, music — needs sampling, spectrograms, and denoising.

Semi-Structured

JSON, XML, log files — flexible schema, common in APIs and NoSQL stores.

Modern pipelines often convert unstructured data into dense numeric embeddings using a pretrained model before it ever reaches a classic ML algorithm — more in Modern Techniques.

The Data Preparation Pipeline



A repeatable pipeline — not a one-off script — is what turns messy raw data into a dependable, production-ready training set.

Inspect early

Know your schema and statistics before writing a single cleaning rule.

Automate

Encode each step as reusable code — pandas or sklearn Pipeline — not manual edits.

Version everything

Track which version of the data produced which model result.

Collect & Inspect Before You Touch the Data

Collection

- APIs and third-party web services
- IoT sensors, logs, and event streams
- Web scraping of public pages
- Public datasets, surveys, and partner data

Inspection

```
df.shape          # row & column counts
df.dtypes         # column data types
df.head()         # quick visual check
df.describe()      # summary statistics
```

Never assume you know your data. A five-minute inspection habit — shape, types, head, describe — catches most surprises before they become bugs deep in a model.

Six Problems Every Raw Dataset Has

Missing Values

NULL / NaN / undefined — typically 5–20% of a dataset

Duplicates

The same record entered more than once, usually from ETL bugs

Wrong Format

Dates stored as text, numbers stored as strings

Outliers / Anomalies

Statistically unexpected, extreme values

Inconsistency

'Tashkent', 'tashkent', 'TASHKENT' — same value, different spellings

Broken Relationships

A foreign key pointing to a record that no longer exists

Missing Values: Understand the Mechanism First

MCAR
Missing Completely At Random

Missingness has no relationship to any variable, observed or not.

Example: *A sensor briefly went offline.*

MAR
Missing At Random

Missingness relates to other variables you can observe.

Example: *Older respondents skip income questions more often.*

MNAR
Missing Not At Random

Missingness depends on the value itself — the hardest case.

Example: *High earners tend to hide their income.*

Filling the Gaps: Imputation Methods

Drop

`dropna()`

Remove the row or column entirely

Mean / Median

`mean()` / `median()`

For numeric columns

Mode

`mode()`

For categorical columns

KNN Imputer

`KNNImputer`

Learn from the nearest neighbors

Iterative Imputer

`IterativeImputer`

Predict each value from other columns (MICE)

Modern approach — deep learning imputers: models such as GAIN (Generative Adversarial Imputation Nets) and datawig learn complex, non-linear patterns across many columns, often outperforming classical imputation on large, high-dimensional datasets.

Duplicates: Exact and Near-Duplicate Records

ID	Name	Age	Email
001	Alice	25	alice@mail.com
002	Bob	30	bob@mail.com
001	Alice	25	alice@mail.com
003	Carla	27	carla@mail.com

Highlighted row = duplicate of row 1

```
df.drop_duplicates(inplace=True)
```

Fuzzy & Near-Duplicates

- Exact duplicates are caught by drop_duplicates() — same values in every column.
- Near-duplicates (typos, reformatted addresses, different capitalization) look different but represent the same real-world entity.
- **Modern approach:** similarity-based record linkage — libraries such as recordlinkage or dedupe score pairs of records and merge likely matches.

Detecting Outliers: The IQR Method

Values that fall more than 1.5× the interquartile range (IQR) beyond the first or third quartile are flagged as statistical outliers — a robust, distribution-free rule of thumb.

- Q1 = 25th percentile, Q3 = 75th percentile
- IQR = Q3 − Q1
- Lower bound = Q1 − 1.5 × IQR
- Upper bound = Q3 + 1.5 × IQR




```
Q1 = df.quantile(0.25)
Q3 = df.quantile(0.75)
IQR = Q3 - Q1
df_clean = df[
    (df >= Q1 - 1.5*IQR) &
    (df <= Q3 + 1.5*IQR)
]
```

Best for

Roughly symmetric numeric data without assuming a normal distribution.

Detecting Outliers: The Z-Score Method

The Z-score measures how many standard deviations a point sits from the mean. Points beyond ±3 standard deviations are typically flagged as outliers.

$$z = (x - \mu) / \sigma$$

- μ = mean of the column, σ = standard deviation
- |z| > 3 is the common cutoff for an outlier
- Assumes data is approximately normally distributed

```
from scipy import stats
import numpy as np

z = np.abs(stats.zscore(df))
df_clean = df[(z < 3).all(axis=1)]
```

Best for

Features that are roughly normally distributed. Sensitive to extreme outliers pulling the mean — combine with IQR when in doubt.

Beyond Statistics: ML-Based Outlier Detection

Isolation Forest

Randomly splits the data — anomalies need far fewer splits to isolate.

```
from sklearn.ensemble import
IsolationForest
model =
IsolationForest(contamination=0.
1)
df['anomaly'] =
model.fit_predict(df)
```

Local Outlier Factor

Density-based — compares a point's local density to its neighbors' density.

```
from sklearn.neighbors import
LocalOutlierFactor
lof =
LocalOutlierFactor(n_neighbors=2
0)
labels = lof.fit_predict(df)
```

Autoencoders

A neural net learns to reconstruct normal data; high reconstruction error flags anomalies — effective in high-dimensional data like images or signals.

Measuring Data Quality: Six Dimensions

Completeness

Are all required values present?

`Available / Expected × 100%`

Accuracy

Do values reflect the real world?

`Correct / Total × 100%`

Consistency

Is the same fact recorded the same way everywhere?

`Consistent / Total × 100%`

Timeliness

Is the data current and up to date?

`Up-to-date / Total × 100%`

Uniqueness

Are there any duplicate records?

`Unique / Total × 100%`

Validity

Do values fall within an allowed range or format?

`Valid format / Total × 100%`

Automating Quality Checks: Profiling Tools

Data profiling automatically analyzes a dataset and produces a detailed report on its quality.

ydata-profiling

- Per-column statistics
- Correlation matrix
- Missing-value visualization
- Interactive HTML report

Great Expectations

- Test data-quality rules
- CI/CD integration
- Automatic documentation
- Data-contract support

Apache Griffin

- Real-time quality monitoring
- Batch and streaming modes
- Hadoop / Spark integration
- Dashboards and alerts

```
from ydata_profiling import ProfileReport
profile = ProfileReport(df)
profile.to_file('report.html')
```

Feature Scaling: Min-Max Normalization

$$x' = (x - \min) / (\max - \min)$$

Output range: [0, 1]

```
from sklearn.preprocessing import
MinMaxScaler

scaler = MinMaxScaler()
df_scaled = scaler.fit_transform(df)
```

Best for

Neural networks, KNN, and SVM — algorithms sensitive to the absolute scale of features.

Why scale at all?

Features measured on very different ranges — age in years vs. income in dollars — can make a model implicitly weight one feature far more than another. Scaling puts every feature on a comparable footing.

Critical rule: fit the scaler ONLY on the training data. On the test set, call transform() only — never fit() again — or information leaks from test into train.

Feature Scaling: Z-Score Standardization

$$x' = (x - \mu) / \sigma$$

Output: mean = 0, standard deviation = 1

```
from sklearn.preprocessing import
StandardScaler

scaler = StandardScaler()
df_scaled = scaler.fit_transform(df)
```

Best for

Linear and logistic regression, PCA, and models that assume normally distributed inputs.

How it differs from Min-Max

Standardization centers data around zero rather than squeezing it into a fixed [0, 1] range — it doesn't bound the output, so it tolerates new values outside the training range more gracefully.

Watch out for

Standardization does not remove outliers — a single extreme value can still distort the mean and standard deviation used in the formula.

Same fit/transform rule applies: fit on train, transform() only on test.

Feature Scaling: Robust Scaling

$$x' = (x - Q2) / (Q3 - Q1)$$

Median-centered; scaled by the interquartile range

```
from sklearn.preprocessing import
RobustScaler

scaler = RobustScaler()
df_scaled = scaler.fit_transform(df)
```

Best for

Datasets with many outliers — median and IQR are far less sensitive to extreme values than mean and standard deviation.

Choosing among the three scalers

Method	Based on	Outlier sensitivity
Min-Max	Min / Max	Very sensitive to outliers
Z-Score	Mean / Std Dev	Sensitive to outliers
Robust	**Median / IQR**	**Resistant to outliers**

Rule of thumb

Start with Standardization for linear models. Switch to Robust Scaling the moment a boxplot or IQR check reveals heavy outliers you don't want to delete.

Encoding Categories: Label vs. One-Hot

Label Encoding

City	Code
Tashkent	0
Samarkand	1
Bukhara	2

```
le = LabelEncoder()
df['city'] = le.fit_transform(df['city'])
```

Best for

Ordinal categories with a real order — e.g. small < medium < large. Using it on unordered categories invents a false numeric relationship.

One-Hot Encoding

City	Tsh	Smk	Bkh
Tashkent	1	0	0
Samarkand	0	1	0
Bukhara	0	0	1

```
pd.get_dummies(df, columns=['city'])
```

Best for

Unordered (nominal) categories with a manageable number of unique values — no ordering is implied between columns.

High-Cardinality Categories: Beyond One-Hot

Target Encoding

Replaces each category with the mean of the target variable for that category.

City	Avg. salary
Tashkent	$85k
Samarkand	$62k
Bukhara	$71k

```
import category_encoders as ce
te = ce.TargetEncoder(cols=['city'])
df = te.fit_transform(df, df['salary'])
```

Best for

High-cardinality columns (hundreds of categories) where one-hot would explode dimensionality.

Modern approach: Entity Embeddings

Instead of hand-picking an encoding, let a neural network learn a dense vector for each category during training. Embeddings automatically capture similarity between categories — e.g. two cities with similar economic profiles end up with similar vectors.

Used heavily in deep tabular models (TabNet, FT-Transformer) and recommendation systems.

Best for

Very high cardinality categories inside a neural network — captures relationships one-hot and target encoding cannot.

When Data Gets Big: The 5 Vs

Volume

Petabytes and beyond — Google processes over 20 PB of data every day.

Velocity

Real-time streams — large platforms ingest hundreds of thousands of events per second.

Variety

Structured, text, image, video — over 80% of it unstructured.

Veracity

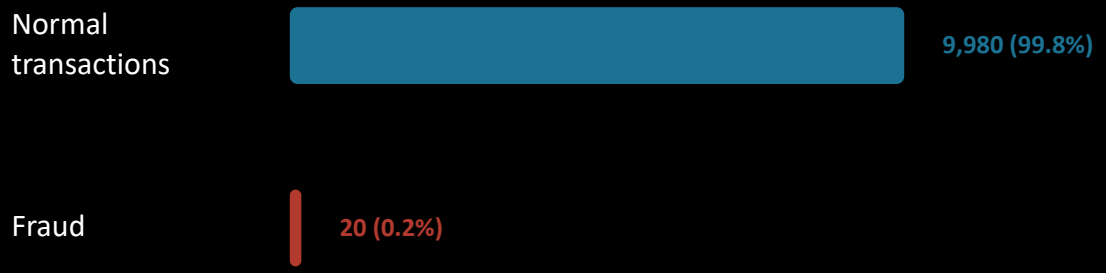
Real-world data is always somewhat noisy, incomplete, or uncertain.

Value

Data only pays off once it is properly cleaned, modeled, and analyzed.

When One Class Dominates: Class Imbalance

Fraud detection example



A model that always predicts "Normal" scores 99.8% accuracy — and is completely useless for catching fraud.

Why accuracy lies here

Standard accuracy rewards ignoring the minority class. For imbalanced problems, evaluate with precision, recall, F1-score, or PR-AUC instead — metrics that expose how well the model handles the rare, usually more important, class.

This is common in fraud detection, disease diagnosis, defect detection, and churn prediction — wherever the event you actually care about is rare.

Rebalancing Classes: SMOTE and Beyond

SMOTE — Synthetic Minority Oversampling Technique

1. Pick a point from the minority class
2. Find its k nearest minority-class neighbors
3. Interpolate a new point between them
4. Add the synthetic sample to the training set

```
from imblearn.over_sampling import SMOTE
sm = SMOTE(random_state=42)
X_res, y_res = sm.fit_resample(X_train,
y_train)
```

Modern alternatives

ADASYN

Adaptive sampling — generates more synthetic points for minority examples that are harder to learn.

Borderline-SMOTE

Focuses new samples near the decision boundary, where they help the model most.

Class weighting / focal loss

Reweight the loss function instead of resampling — e.g. class_weight='balanced'.

⚠ SMOTE only on TRAINING data — never resample the test set.

Generating More Data: Image Augmentation

Classic techniques (e.g. Albumentations library)

- Rotation ±30°
- Horizontal flip
- Random crop
- Brightness / contrast shift
- Gaussian noise
- Zoom in / out

Modern techniques

- **AutoAugment / RandAugment** — search algorithms that learn the best augmentation policy for a dataset automatically.
- **Mixup & CutMix** — blend two images (and their labels) to create new, harder training examples.
- **Diffusion-based synthesis** — models like Stable Diffusion generate entirely new, realistic training images.

Generating More Data: Audio Augmentation

Time Stretching

Speed audio up or down without changing pitch

Pitch Shift

Raise or lower the tone of a recording

Background Noise

Mix in ambient or environmental noise

Volume Change

Randomly scale loudness

SpecAugment

Mask random time and frequency bands on the spectrogram

Reverb / Echo

Simulate different room acoustics

Feature Engineering: Creating Signal from Raw Data

Time Features

Day, month, year, hour, weekday, season, is_weekend flags.

Example: a food-delivery model learns Friday evenings are busier.

Mathematical Transforms

log(x), sqrt(x), x^2, and binning continuous values into categories.

Example: log(income) tames a heavily skewed salary distribution.

Combined Features

Ratios, products, differences, and group-level aggregations.

Example: debt_to_income = debt ÷ income often beats either column alone.

Text Features

Word count, character length, keyword flags, sentiment score.

Example: review length and sentiment both help predict a 1-star review.

Feature Selection: Keeping What Matters

Filter Methods

Model-independent — score each feature on its own.

- Pearson correlation
- Chi-square test
- ANOVA F-test
- Mutual information

Wrapper Methods

Retrain the model repeatedly to test feature subsets.

- Recursive Feature Elimination
- Forward / backward selection
- Exhaustive search
- BORUTA algorithm

Embedded Methods

Selection happens inside model training itself.

- LASSO (L1 regularization)
- Ridge (L2 regularization)
- Random Forest importance
- Gradient Boosting importance

```
from sklearn.feature_selection import SelectKBest, f_classif
selector = SelectKBest(f_classif, k=10)
X_new = selector.fit_transform(X, y)
```

Data-Centric AI: A Shift in Mindset

Popularized by Andrew Ng: instead of endlessly tuning model architecture on a fixed dataset, systematically and iteratively improve the data itself — fix mislabeled examples, add missing edge cases, remove noisy records.

Common tools

- **Cleanlab** — automatically finds likely label errors in a dataset
- **Snorkel** — programmatic, rule-based labeling at scale
- **Encord / Labelbox** — platforms for managing data quality and annotation

> *"The most effective ML teams spend more time on their data than on their models."*

Model-Centric vs. Data-Centric

Model-Centric	Data-Centric
Fixed dataset	Fixed(ish) architecture
Iterate on architecture & hyperparameters	Iterate on data quality, labels, coverage
Diminishing returns at scale	Often the highest-leverage lever available

In practice

Run error analysis on what the model gets wrong, trace those errors back to specific data problems, fix just that slice, and retrain — a tight loop that often beats a bigger model.

Synthetic Data: Generating What You Don't Have

GANs / VAEs

Generate realistic synthetic images and tabular rows — e.g. CTGAN for structured, tabular data.

LLM-Generated Data

Prompt GPT-4 or Claude to synthesize labeled text examples, instructions, or rare edge cases.

Simulation

Physics and game engines generate labeled data for robotics and autonomous driving.

Privacy-Preserving Synthesis

Differential privacy lets teams share realistic data without exposing real records.

Let Foundation Models Do the Feature Engineering

Instead of hand-crafting features from text, images, or audio, pass raw data through a pretrained model and use its output vector as the feature representation.

Text

`BERT / Sentence-Transformers`

Images

`CLIP / ResNet`

Audio

`Whisper / wav2vec`

Why it matters: transfer learning from a foundation model drastically reduces how much labeled data you need to reach strong performance. Store the resulting vectors in a vector database (FAISS, Pinecone, Weaviate) for fast similarity search and retrieval.

Keeping Data Pipelines Trustworthy at Scale

Automated Validation

Great Expectations, Evidently AI, Deequ — quality gates built into CI/CD pipelines.

Blocks a bad batch of data from ever reaching a training job.

Data Contracts

Agreed-upon schemas and expectations between data producers and consumers.

Breaks loudly when an upstream team changes a column, instead of silently.

Feature Stores

Feast, Tecton — consistent features across training and serving, preventing skew.

The #1 cause of "it worked in training but not in production" bugs.

Data Versioning

DVC, MLflow — reproducible datasets tied to every model experiment.

Lets you answer: which exact data produced this model?

The Golden Rule: Fit on Train, Transform on Test

Splitting your data

- 80/20 or 70/30 train/test split is a common starting point
- Stratified split preserves class ratios in imbalanced data
- Time-based split (no shuffling) for time series and forecasting
- k-fold cross-validation gives a more robust performance estimate

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42)

scaler.fit(X_train)              # fit on TRAIN only
X_train = scaler.transform(X_train)
X_test  = scaler.transform(X_test)    # transform only
```

Data Leakage

Any scaler, encoder, imputer, or SMOTE call must call fit() only on the training set.

Fitting on test data lets information leak backward from test into train — your evaluation metrics look great, but the model fails in production.

Time series?

Never shuffle. Split chronologically so the model is only ever tested on data that comes after its training window.

Choosing the Right Tool for Your Data Size

Library	Data Size	Speed	Best For
pandas	< 1 GB	★★★☆☆	Small–medium data, fast prototyping
Dask	1–100 GB	★★★★☆	Parallelizing existing pandas code
Polars	< 50 GB	★★★★★	Raw speed — 5–10x faster than pandas
PySpark	100 GB+	★★★★☆	Enterprise big data, streaming, Hadoop
DuckDB	< 100 GB	★★★★★	SQL directly on CSV / Parquet files
SQLAlchemy	Database	★★★☆☆	Database integration, ORM, SQL builder

Common Mistakes — and How to Avoid Them

Fitting a scaler or encoder on test data	Fit only on train; call transform() on test	*Prevents data leakage that inflates results*
Dropping every row with a missing value	Diagnose the missingness mechanism, then choose a targeted strategy	*Avoids throwing away valuable data*
Label-encoding unordered categories	Use one-hot encoding, or an ordinal encoder only when order is real	*Avoids inventing a false numeric relationship*
Applying SMOTE to the test set	Resample the training data only	*Keeps evaluation results realistic*

Data Readiness Checklist

Initial Inspection

- df.shape, df.dtypes
- df.head() / df.describe()
- Note units and expected ranges

Missing Values

- isnull().sum() and % missing
- Diagnose MCAR / MAR / MNAR
- Drop or impute deliberately

Duplicates

- duplicated().sum()
- drop_duplicates()
- Check near-duplicates too

Outliers

- IQR or Z-score check
- Boxplot visualization
- Decide: fix, cap, or keep

Consistency

- nunique(), value_counts()
- Normalize text formatting
- Validate foreign-key relationships

Model Readiness

- Encode categoricals, scale features
- Fit scalers/encoders on train only
- Train / test split, cross-validation




THANK YOU